Worksheet EDA1 - WS05: Alocação Dinâmica I

Conteúdo: malloc, calloc, NULL e representação de memória | Tempo previsto: 30-35 minutos | Modalidade: individual ou dupla

Objetivos de aprendizagem

- Distinguir uma variável ponteiro do bloco de memória para o qual ela aponta.
- Interpretar chamadas a malloc e calloc.
- Representar estados da memória antes e depois da alocação.
- Reconhecer a necessidade de verificar NULL.
- Rastrear acessos a elementos de um bloco alocado.
- Identificar erros de dimensionamento e de leitura de valores não inicializados.

Parte A - Compreensão

1. Complete:

a) A alocação dinâmica permite decidir a quantidade de memória durante _____.

b) A função _____ solicita um número total de bytes e não inicializa os elementos.

c) A função _____ recebe quantidade e tamanho de cada elemento e inicializa o bloco com zeros.

d) Se uma alocação falhar, o valor retornado é _____.

2. Marque V ou F e justifique as falsas:

( ) malloc(4 * sizeof(int)) solicita espaço para quatro objetos do tipo int.

( ) Após malloc, todos os inteiros do bloco possuem automaticamente valor 0.

( ) Após calloc(4, sizeof(int)), os quatro inteiros começam com valor 0.

( ) Se p == NULL, não devemos acessar p[0].

( ) A variável ponteiro p e o bloco alocado ocupam necessariamente a mesma região de memória.

_____

Parte B - Simulação e representação

3. Considere endereços simbólicos. Antes da alocação, p está no endereço 1000.

```
int *p;

p = malloc(3 * sizeof(int));
```

Suponha que a alocação tenha sucesso e o endereço inicial do bloco retornado seja 3000.

a) Qual valor p armazena após a atribuição? _____

b) Quantos elementos int existem no bloco? _____

c) Podemos afirmar os valores iniciais de p[0], p[1] e p[2]? Explique.

_____

d) Desenhe p separado do bloco e use uma seta para representar a relação.

_____

_____

_____

4. Agora considere:

```
int *q;
q = calloc(3, sizeof(int));
if (q != NULL)
    q[1] = 7;
```

Supondo sucesso, preencha:

Estado	q[0]	q[1]	q[2]
final			

Parte C - Leitura de código

5. Analise:

```
int *v;
int i;
v = malloc(4 * sizeof(int));
if (v != NULL)
{
    for (i = 0; i < 4; i++)
        v[i] = i * 2;
}
```

Supondo sucesso da alocação:

a) v[0] = _____ v[1] = _____ v[2] = _____ v[3] = _____

b) O acesso v[4] estaria dentro do bloco alocado? _____

c) Qual condição impede o laço de acessar v[4]?

_____

6. Identifique o problema:

```
int n = 5;
int *v;
v = malloc(n);
```

O objetivo era alocar espaço para cinco inteiros. Reescreva corretamente a chamada.

_____

7. Considere:

```
int *v;
v = malloc(3 * sizeof(int));
if (v != NULL)
    printf("%d\n", v[0]);
```

Por que não é correto prever um valor específico para o que será impresso?

_____

Parte D - Construção

8. Complete para alocar n inteiros com malloc e tratar falha:

```
int *v;
v = ____________________________________________;
if (_______________________________)
{
    printf("Falha na alocacao.\n");
    return 1;
}
```

9. Complete para alocar n inteiros inicialmente zerados:

```
int *v;
v = ____________________________________________;
```

Parte E - Problema integrador

10. Considere que n == 4 e que a alocação abaixo retorna o endereço simbólico 5000:

```
int *dados;
dados = calloc(n, sizeof(int));
if (dados != NULL)
{
    dados[0] = 9;
    dados[3] = 2;
}
```

a) Represente dados e o bloco alocado em um diagrama de memória.

_____

_____

_____

b) Escreva o estado final dos quatro elementos:

dados[0] = _____ dados[1] = _____ dados[2] = _____ dados[3] = _____

c) Explique por que dados[1] e dados[2] têm valores conhecidos mesmo sem atribuições explícitas.

_____

Preparação para a próxima aula

Na próxima aula estudaremos o uso completo da memória alocada e sua liberação. Antes de sair, certifique-se de que consegue explicar:

1. a diferença entre o ponteiro e o bloco apontado;
2. por que malloc e calloc podem retornar NULL;
3. como calcular o tamanho de um bloco para n inteiros;
4. qual a diferença entre o conteúdo inicial produzido por malloc e por calloc.